CO3 – AT-3

Policy Optimization Assessment

ASSESSMENT - 3

Data Interpretation – PPO, TRPO & DDPG Performance Analysis

N. PARNITA

192525322

CODE:

```
!pip install -q gymnasium[classic-control] stable-baselines3
import gymnasium as gym
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from stable_baselines3 import PPO, DDPG
!pip install -q sb3-contrib
from sb3_contrib import TRPO

env = gym.make("Pendulum-v1")

print("Environment: Pendulum-v1")
print("Action Space:", env.action_space)
print("Observation Space:", env.observation_space)

ppo = PPO(
    "MlpPolicy",
    env,
    learning_rate=0.0003,
    verbose=0
)

ppo.learn(total_timesteps=20000)

print("PPO training completed!")
trpo = TRPO(
    "MlpPolicy",
    env,
    learning_rate=0.0003,
    verbose=0
)

trpo.learn(total_timesteps=20000)

print("TRPO training completed!")
ddpg = DDPG(
    "MlpPolicy",
    env,
```

```
    learning_rate=0.001,
    buffer_size=1000,
    learning_starts=100,
    batch_size=32,
    verbose=0
)

ddpg.learn(total_timesteps=1000)

print("DDPG training completed!")

def evaluate(model, episodes=5):
    scores = []

    for _ in range(episodes):
        state, _ = env.reset()
        total_reward = 0
        done = False

        while not done:
            action, _ = model.predict(state, deterministic=True)

            state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            total_reward += reward

        scores.append(total_reward)

    return np.mean(scores)


ppo_score = evaluate(ppo)
trpo_score = evaluate(trpo)
ddpg_score = evaluate(ddpg)

print("PPO Average Reward :", round(ppo_score, 2))
print("TRPO Average Reward:", round(trpo_score, 2))
print("DDPG Average Reward:", round(ddpg_score, 2))

results = pd.DataFrame({
    "Algorithm": ["PPO", "TRPO", "DDPG"],
```

```
    "Average Reward": [ppo_score, trpo_score, ddpg_score]
})

print("PERFORMANCE COMPARISON")
display(results)

# Bar graph
plt.figure(figsize=(8, 5))
plt.bar(results["Algorithm"], results["Average Reward"])

plt.xlabel("Algorithm")
plt.ylabel("Average Reward")
plt.title("PPO vs TRPO vs DDPG Performance")
plt.grid(axis="y")
plt.show()

# Best algorithm
best = results.loc[results["Average Reward"].idxmax(), "Algorithm"]

print("\nEVALUATION")
print("Best performing algorithm:", best)
print("PPO, TRPO and DDPG were evaluated using the same environment.")
print("Higher average reward indicates better performance.")
```

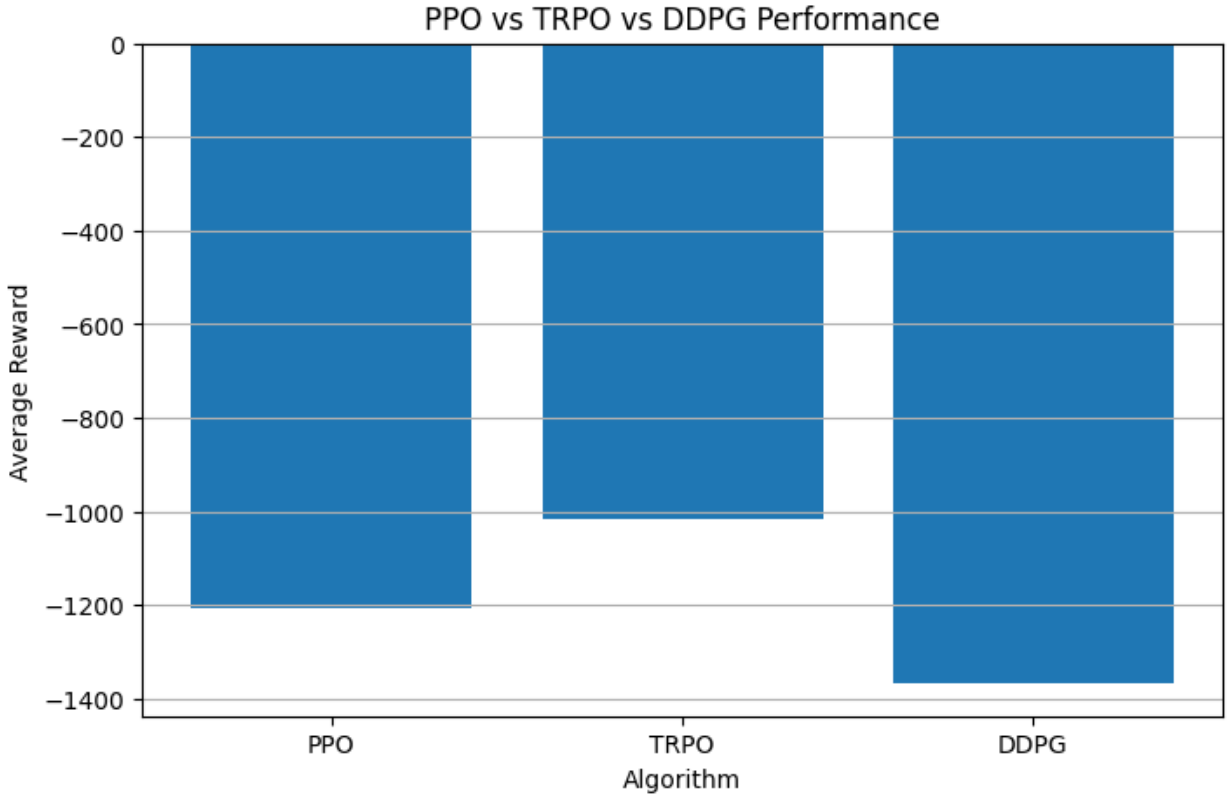
OUTPUT:

```
Environment: Pendulum-v1
Action Space: Box(-2.0, 2.0, (1,), float32)
Observation Space: Box([-1. -1. -8.], [1. 1.
8.], (3,), float32)

PPO training completed!
TRPO training completed!
DDPG training completed!
```

PPO Average Reward : -1208.03
TRPO Average Reward: -1015.4
DDPG Average Reward: -1368.96
PERFORMANCE COMPARISON

	Algorithm	Average Reward
0	PPO	-1208.031949
1	TRPO	-1015.397880
2	DDPG	-1368.962796



EVALUATION
Best performing algorithm: TRPO

PPO, TRPO and DDPG were evaluated using the same environment.
Higher average reward indicates better performance.